Lab Manual

Programming for AI

By

AMNA SHAHZAD **2023-BS-AI-027**

Submitted to

Mr. SAMRAIZ

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCES

TABLE OF CONTENTS

LAB NO 01

INTRODUCTION TO PYTHON

Python in Artificial Intelligence (AI)

- Artificial Intelligence (AI) enables machines to **simulate human intelligence** such as learning, reasoning, and decision-making.
- Python is the most widely used programming language in AI due to its simplicity and powerful libraries.

Why Python is Used in AI

- Simple and readable syntax.
- Large collection of AI libraries.
- Strong community support.
- Platform independent.
- Rapid development and prototyping.

Role of Python in AI

- Data preprocessing and analysis.
- Feature extraction.
- Model training and testing.
- Prediction and decision-making.
- Visualization of results

Important Python Libraries for AI

NumPy

- Used for numerical computations.
- Supports multi-dimensional arrays and matrices.
- Fast mathematical operations

Syntax:

import numpy as np

Pandas

- Used for data manipulation and analysis.
- Handles large datasets efficiently.

import pandas as pd

Matplotlib & Seaborn

- Used for data visualization.
- Helps understand patterns and trends.

import matplotlib.pyplot as plt

import seaborn as sns

Scikit-learn

- Provides machine learning algorithms.
- Used for classification, regression, and clustering.

from sklearn.model_selection import train_test_split

TensorFlow

- Used for deep learning and neural networks.

import tensorflow as tf

PyTorch

- Popular for research and deep learning applications.

import torch

OS Library

- Used for file and directory handling.

import os

Applications of Python in AI

- Speech recognition.
- Face and emotion recognition.
- Chatbots and virtual assistants.
- Recommendation systems.
- Medical diagnosis.
- Autonomous vehicles

Advantages of Python in AI

- Easy integration with other languages.
- Large ecosystem of tools and frameworks.
- Suitable for both beginners and professionals.
- Open-source and cost-effective.

Limitations of Python in AI

- Slower execution compared to C/C++.
- High memory consumption.
- Not ideal for mobile applications directly.

LAB EXERCISE

Program 1: Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal).

Banking Transaction Validator

Code:

PROGRAM 1

[2]:
```
# ATM Simulation with PIN Validation and Withdrawal

# Predefined correct PIN and initial balance
correct_pin = "1234"
balance = 10000.0  # Initial account balance
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    entered_pin = input("Enter your 4-digit PIN: ")

    if entered_pin == correct_pin:
        print(f"\n PIN verified successfully.")
        print(f" Current Balance: PKR {balance:.2f}")

        try:
            withdrawal = float(input("Enter withdrawal amount: PKR "))

            if withdrawal <= 0:
                print(" Invalid amount. Must be greater than zero.")
                break

            service_charge = withdrawal * 0.02
            total_deduction = withdrawal + service_charge

            if total_deduction > balance:
                print(" Insufficient funds including service charge.")
            else:
                balance -= total_deduction
                print(f"\nWithdrawal successful.")
                print(f" Amount withdrawn: PKR {withdrawal:.2f}")
                print(f" Service charge (2%): PKR {service_charge:.2f}")
                print(f" Updated Balance: PKR {balance:.2f}")
            break

        except ValueError:
            print(" Invalid input. Please enter a numeric amount.")
            break

    else:
        attempts += 1
        print(f" Incorrect PIN. Attempts left: {max_attempts - attempts}")
        if attempts == max_attempts:
            print(" Card blocked due to 3 invalid attempts.")
```

Output:

```
Enter your 4-digit PIN:  1234

 PIN verified successfully.
 Current Balance: PKR 10000.00
Enter withdrawal amount: PKR  5689

Withdrawal successful.
 Amount withdrawn: PKR 5689.00
 Service charge (2%): PKR 113.78
 Updated Balance: PKR 4197.22
```

Program 2: Grocery Store Inventory System
Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.
If stock available, deduct from inventory.
Apply discount slabs (10%, 15%, 20%).
Print final bill with tax (5%).

Grocery Store Inventory System

Code:

PROGRAM 2

[9]:
```
# Grocery Store Inventory System

# Inventory: item_name → {price, stock}
inventory = {
    "Apples": {"price": 200, "stock": 50},
    "Bananas": {"price": 100, "stock": 100},
    "Milk": {"price": 250, "stock": 30},
    "Bread": {"price": 150, "stock": 40},
    "Eggs": {"price": 20, "stock": 200}
}

cart = {}
total = 0

print("Welcome to the Grocery Store!\nAvailable items:")
for item, details in inventory.items():
    print(f"- {item}: PKR {details['price']} (Stock: {details['stock']})")

while True:
    item = input("\nEnter item name to purchase (or 'done' to finish): ").title()
    if item == "Done":
        break
    if item not in inventory:
        print(" Item not found.")
        continue

    try:
        qty = int(input(f"Enter quantity of {item}: "))
        if qty <= 0:
            print(" Quantity must be positive.")
            continue
        if qty > inventory[item]["stock"]:
            print(" Not enough stock available.")
            continue

        price = inventory[item]["price"]
        subtotal = price * qty
        cart[item] = {"qty": qty, "subtotal": subtotal}
        total += subtotal
        inventory[item]["stock"] -= qty
        print(f" Added {qty} x {item} to cart. Subtotal: PKR {subtotal}")
```

```
        total += subtotal
        inventory[item]["stock"] -= qty
        print(f" Added {qty} x {item} to cart. Subtotal: PKR {subtotal}")

    except ValueError:
        print(" Invalid quantity. Please enter a number.")

# Apply discount slabs
discount = 0
if total >= 5000:
    discount = 0.20
elif total >= 3000:
    discount = 0.15
elif total >= 1000:
    discount = 0.10

discount_amount = total * discount
tax = (total - discount_amount) * 0.05
final_total = total - discount_amount + tax

# Print final bill
print("\n Final Bill:")
for item, details in cart.items():
    print(f"{item} x {details['qty']} → PKR {details['subtotal']}")

print(f"\n Total: PKR {total:.2f}")
print(f" Discount ({int(discount*100)}%): -PKR {discount_amount:.2f}")
print(f" Tax (5%): +PKR {tax:.2f}")
print(f" Final Amount Payable: PKR {final_total:.2f}")
```

Output:

```
Welcome to the Grocery Store!
Available items:
- Apples: PKR 200 (Stock: 50)
- Bananas: PKR 100 (Stock: 100)
- Milk: PKR 250 (Stock: 30)
- Bread: PKR 150 (Stock: 40)
- Eggs: PKR 20 (Stock: 200)

Enter item name to purchase (or 'done' to finish):  done

 Final Bill:

 Total: PKR 0.00
 Discount (0%): -PKR 0.00
 Tax (5%): +PKR 0.00
 Final Amount Payable: PKR 0.00
```

Program 3: Student Grading with Subject Weightage
Input marks for 5 subjects where each subject has different weightage (stored in a tuple).
Compute weighted average.
Assign grade (A/B/C/D/Fail).
Also, display whether the student is eligible for scholarship (≥80% weighted).

Student Grading with Subject Weightage

Code:

PROGRAM 3

```
[12]: # Subject weightages (in percentage)
# Example: Math - 30%, Physics - 25%, Chemistry - 20%, English - 15%, Computer - 10%
weightages = (0.30, 0.25, 0.20, 0.15, 0.10)

subjects = ["Math", "Physics", "Chemistry", "English", "Computer"]
marks = []

print(" Enter marks for the following subjects (out of 100):")
for subject in subjects:
    while True:
        try:
            score = float(input(f"{subject}: "))
            if 0 <= score <= 100:
                marks.append(score)
                break
            else:
                print(" Marks must be between 0 and 100.")
        except ValueError:
            print(" Invalid input. Please enter a number.")

# Calculate weighted average
weighted_total = sum(m * w for m, w in zip(marks, weightages))

# Assign grade
if weighted_total >= 90:
    grade = "A"
elif weighted_total >= 80:
    grade = "B"
elif weighted_total >= 70:
    grade = "C"
elif weighted_total >= 60:
    grade = "D"
else:
    grade = "Fail"

# Scholarship eligibility
eligible = weighted_total >= 80

# Display results
print("\n Result Summary:")
print(f"Weighted Average: {weighted_total:.2f}%")
print(f"Grade: {grade}")
print(f" Scholarship Eligibility: {' Eligible' if eligible else ' Not Eligible'}")
```

Output

```
 Enter marks for the following subjects (out of 100):
Math:  56
Physics:  34
Chemistry:  67
English:  89
Computer:  78

 Result Summary:
Weighted Average: 59.85%
Grade: Fail
 Scholarship Eligibility:  Not Eligible
```

Program 4: BMI + Health Recommendation
Extend BMI program:
Take weight, height, and age.
Compute BMI.
Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

BMI + Health Recommendation

Code:

PROGRAM 4

[14]:
```
# BMI + Health Recommendation

def get_bmi(weight, height_cm):
    height_m = height_cm / 100
    return weight / (height_m ** 2)

def get_bmi_category(bmi):
    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

def health_recommendation(bmi_category, age):
    if bmi_category == "Underweight":
        if age < 18:
            return " Eat calorie-dense foods, include dairy and nuts.  Light strength training recommended."
        elif age <= 40:
            return " Add protein-rich meals, healthy fats, and resistance workouts."
        else:
            return " Focus on balanced nutrition and low-impact exercises like walking or yoga."

    elif bmi_category == "Normal":
        return " Maintain current lifestyle.  Balanced diet and regular physical activity."

    elif bmi_category == "Overweight":
        if age < 18:
            return " Encourage outdoor play, reduce sugary snacks.  More fruits and veggies."
        elif age <= 40:
            return " Moderate exercise 4–5 times/week.  Low-carb, high-fiber diet."
        else:
            return " Daily walks, portion control, and regular health checkups."

    elif bmi_category == "Obese":
        if age < 18:
            return " Consult a pediatrician.  Controlled diet and active lifestyle."
        elif age <= 40:
            return " Follow a structured weight-loss plan.  High-fiber, low-fat meals.  Cardio + strength training."
        else:
            return " Gentle workouts, medical guidance, and a heart-healthy diet."
```

```
        elif age <= 40:
            return " Moderate exercise 4–5 times/week.  Low-carb, high-fiber diet."
        else:
            return " Daily walks, portion control, and regular health checkups."

    elif bmi_category == "Obese":
        if age < 18:
            return " Consult a pediatrician.  Controlled diet and active lifestyle."
        elif age <= 40:
            return " Follow a structured weight-loss plan.  High-fiber, low-fat meals.  Cardio + strength training."
        else:
            return " Gentle workouts, medical guidance, and a heart-healthy diet."

# Input
try:
    weight = float(input("Enter your weight (kg): "))
    height = float(input("Enter your height (cm): "))
    age = int(input("Enter your age: "))

    bmi = get_bmi(weight, height)
    category = get_bmi_category(bmi)
    recommendation = health_recommendation(category, age)

    print(f"\n📊 Your BMI: {bmi:.2f} → {category}")
    print(f" Age: {age}")
    print(f"\n Health Recommendation:\n{recommendation}")

except ValueError:
    print(" Invalid input. Please enter numeric values.")
```

Output:

```
Enter your weight (kg):  60
Enter your height (cm):  670
Enter your age:  21

📊 Your BMI: 1.34 → Underweight
Age: 21

Health Recommendation:
Add protein-rich meals, healthy fats, and resistance workouts.
```

Program 5: Telecom Prepaid Recharge System
Maintain call rate/min, SMS rate, and data/MB in a dictionary.
User selects usage (calls, SMS, data).
Deduct accordingly from balance.
If balance < 20% of initial → print recharge reminder.
Apply bonus balance if recharge > 1000.

Telecom Prepaid Recharge System

Code:

PROGRAM 5

[16]:
```
# Telecom Prepaid Recharge System

# Rates per unit
rates = {
    "call": 2.0,     # PKR per minute
    "sms": 0.5,      # PKR per SMS
    "data": 1.0      # PKR per MB
}

# Initial recharge
initial_balance = float(input("Enter recharge amount (PKR): "))
balance = initial_balance

# Apply bonus if recharge > 1000
if initial_balance > 1000:
    bonus = initial_balance * 0.10
    balance += bonus
    print(f" Bonus applied: PKR {bonus:.2f}")
else:
    bonus = 0

print(f" Total Balance: PKR {balance:.2f}")

# Usage loop
while True:
    print("\nSelect usage type: call / sms / data / exit")
    usage = input("Your choice: ").lower()

    if usage == "exit":
        break
    if usage not in rates:
        print(" Invalid option.")
        continue

    try:
        units = float(input(f"Enter number of {usage}s: "))
        if units <= 0:
            print(" Must be greater than zero.")
            continue
```

```
            print(" Must be greater than zero.")
            continue

        cost = units * rates[usage]
        if cost > balance:
            print(" Insufficient balance.")
            continue

        balance -= cost
        print(f" {units} {usage}(s) used. Cost: PKR {cost:.2f}")
        print(f" Remaining Balance: PKR {balance:.2f}")

        # Recharge reminder
        if balance < 0.20 * initial_balance:
            print(" Low balance! Please recharge soon.")

    except ValueError:
        print(" Invalid input. Please enter a number.")

# Final summary
print("\n Session Summary:")
print(f"Initial Recharge: PKR {initial_balance:.2f}")
print(f"Bonus Applied: PKR {bonus:.2f}")
print(f"Remaining Balance: PKR {balance:.2f}")
```

Output

```
Enter recharge amount (PKR):  20000
 Bonus applied: PKR 2000.00
 Total Balance: PKR 22000.00

Select usage type: call / sms / data / exit
Your choice:  call
Enter number of calls:  03324567890
 Insufficient balance.

Select usage type: call / sms / data / exit
Your choice:  exit

 Session Summary:
Initial Recharge: PKR 20000.00
Bonus Applied: PKR 2000.00
Remaining Balance: PKR 22000.00
```

LAB NO 02

LAB EXERCISE

Program 6: E-Commerce Discount Engine with Membership Levels
User inputs bill amount + membership type (Regular, Gold, Platinum).
Apply different discount rules (Gold +10%, Platinum +15%).
If bill > 10000, apply additional 5% flat.
Show itemized receipt with tax breakdown.

E-Commerce Discount Engine with Membership Levels

Code:

PROGRAM 6

[8]:
```
import pandas as pd
import numpy as np
import re
import os
```

12]:
```
bill = float(input("Enter bill amount: "))
member = input("Membership (Regular/Gold/Platinum): ").lower()

discount = 0
if member == "gold":
    discount = 0.10
elif member == "platinum":
    discount = 0.15

bill_after_discount = bill - (bill * discount)

if bill > 10000:
    bill_after_discount -= bill_after_discount * 0.05

tax = bill_after_discount * 0.08
final_amount = bill_after_discount + tax

print("Final Amount:", final_amount)
print("Tax:", tax)
```

Output:

```
Enter bill amount:  2000
Membership (Regular/Gold/Platinum):  Gold
Final Amount: 1944.0
Tax: 144.0
```

Program 7: Hospital Emergency Unit with Multi-Condition Triage
Take patient details: temperature, pulse, oxygen, blood pressure.
Classify into categories: Stable, Observation, Urgent, Critical.
If critical → suggest "Admit in ICU."
If urgent → suggest "Immediate doctor attention."
Use nested conditions.

Hospital Emergency Unit

Code:

PROGRAM 7

```
[15]: temp = float(input("Temperature: "))
      pulse = int(input("Pulse: "))
      oxygen = int(input("Oxygen level: "))
      bp = int(input("Blood Pressure: "))

      if oxygen < 85 or bp < 80:
          print("Critical → Admit in ICU")
      elif temp > 39 or pulse > 120:
          print("Urgent → Immediate doctor attention")
      elif temp > 37:
          print("Observation")
      else:
          print("Stable")
```

Output:

```
Temperature:  45
Pulse:  72
Oxygen level:  63
Blood Pressure:  112
Critical → Admit in ICU
```

Program 8: Online Examination Grader with Negative Marking
Take total marks, obtained marks, and number of wrong answers.
Deduct 0.25 per wrong answer.
Compute percentage.
Assign division + eligibility for scholarship.
If Fail → suggest "Repeat Exam."

Code:

Online Examination Grader

PROGRAM 8

```
[20]: total = int(input("Total Marks: "))
obtained = int(input("Obtained Marks: "))
wrong = int(input("Wrong Answers: "))

obtained -= wrong * 0.25
percentage = (obtained / total) * 100

if percentage >= 80:
    division = "Distinction + Scholarship"
elif percentage >= 60:
    division = "First Division"
elif percentage >= 40:
    division = "Pass"
else:
    division = "Fail → Repeat Exam"

print("Percentage:", percentage)
print("Result:", division)
```

Output:

```
Total Marks:  100
Obtained Marks:  88
Wrong Answers:  12
Percentage: 85.0
Result: Distinction + Scholarship
```

Program 9: Movie Ticket Booking with Age & Timing Policy
User inputs: age, showtime, ticket type (Normal/3D/IMAX).
Apply child restrictions (No late-night for <12).
Apply senior citizen discount (30%).
Apply peak hour charges (10%).
Print final ticket price.

Movie Ticket Booking

Code:

PROGRAM 9

```
[23]: age = int(input("Age: "))
      time = int(input("Showtime (24h): "))
      ticket = input("Type (Normal/3D/IMAX): ").lower()

      price = {"normal": 500, "3d": 800, "imax": 1200}[ticket]

      if age < 12 and time > 20:
          print("Late night show not allowed")
      else:
          if age >= 60:
              price *= 0.7
          if 18 <= time <= 22:
              price *= 1.1

          print("Final Ticket Price:", price)
```

Output:

```
Age:  21
Showtime (24h):  22
Type (Normal/3D/IMAX):  3D
Final Ticket Price: 880.0000000000001
```

Program 10: Weather-based Outfit + Travel Suggestion
Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).
Suggest outfit + accessories.
If Rainy → umbrella; If Cold → gloves.
If event is Business + temperature > 35 → recommend "light formal suit."

Weather-based Outfit + Travel Suggestion

Code:

PROGRAM 10

[26]:
```
temp = int(input("Temperature: "))
weather = input("Weather: ").lower()
event = input("Event Type: ").lower()

if event == "business" and temp > 35:
    print("Light formal suit")

if weather == "rainy":
    print("Carry umbrella")
elif weather == "cold":
    print("Wear gloves")
```

Output:

```
Temperature:  77
Weather:  Rainy
Event Type:  business
Light formal suit
Carry umbrella
```

LAB NO 03

LAB EXERCISE

Program 11: ATM Withdrawal with Note Counting + Receipt
Take withdrawal amount.
Compute minimum number of notes (1000, 500, 100, 50, 10, 1).
Also compute service charge = 0.5%.
Print receipt with remaining balance after deduction.

ATM Withdrawal with Note Counting

Code:

PROGRAM 11

```
[29]: amount = int(input("Withdraw Amount: "))
      balance = 50000

      notes = [1000, 500, 100, 50, 10, 1]
      for n in notes:
          print(n, ":", amount // n)
          amount %= n

      charge = balance * 0.005
      balance -= charge

      print("Service Charge:", charge)
      print("Remaining Balance:", balance)
```

Output:

```
Withdraw Amount:  5000
1000 : 5
500 : 0
100 : 0
50 : 0
10 : 0
1 : 0
Service Charge: 250.0
Remaining Balance: 49750.0
```

Program 12: Library Fine Calculator with Membership Levels
Input days late.
Apply fines (10, 20, 50/day).

If days > 30 → "Membership Cancelled."
If user is Premium member, reduce fine by 50%.
Print detailed fine slip

Library Fine Calculator

Code:

PROGRAM 12

```
[32]: days = int(input("Days Late: "))
      member = input("Membership (Normal/Premium): ").lower()

      rate = 10 if days <= 7 else 20 if days <= 30 else 50
      fine = days * rate

      if member == "premium":
          fine *= 0.5
      if days > 30:
          print("Membership Cancelled")

      print("Total Fine:", fine)
```

Output:

```
Days Late:  20
Membership (Normal/Premium):  Normal
Total Fine: 400
```

Program 13: Sales Analysis Dashboard
Store monthly sales in a list (12 entries).
Print max, min, average.
Print months above average.
Use loop + condition to print "Growth" or "Decline" compared to last month.

Sales Analysis Dashboard

Code:

PROGRAM 13

```
[37]: sales = [int(input(f"Month {i+1}: ")) for i in range(12)]

      avg = sum(sales) / 12
      print("Max:", max(sales), "Min:", min(sales), "Average:", avg)

      for i in range(1, 12):
          print("Growth" if sales[i] > sales[i-1] else "Decline")
```

Output:

```
Month 1:  6000
Month 2:  5000
Month 3:  4000
Month 4:  3000
Month 5:  2000
Month 6:  1000
Month 7:  500
Month 8:  200
Month 9:  300
Month 10:  250
Month 11:  1150
Month 12:  2500
Max: 6000 Min: 200 Average: 2158.3333333333335
Decline
Decline
Decline
Decline
Decline
Decline
Decline
Growth
Decline
Growth
Growth
```

Program 14: Multiplication Puzzle with Random Blanks
Generate a multiplication table (1–10).
Randomly replace 3 results with "??".
Ask user to guess values.
Track score.
Give performance remark (Excellent/Good/Try Again).

Multiplication Puzzle with Random Blanks

Code:

PROGRAM 14

[39]:
```python
import random

score = 0
for i in range(1, 11):
    for j in range(1, 11):
        if random.randint(1, 20) == 5:
            ans = int(input(f"{i} x {j} = ?? "))
            if ans == i * j:
                score += 1

print("Score:", score)
```

Output:

```
5 x 2 = ??  10
6 x 10 = ??  60
8 x 4 = ??  32
9 x 8 = ??  72
10 x 4 = ??  40
Score: 5
```

Program 15: Password Strength + Re-entry Attempts
Check password rules (length, digit, upper, lower, special).
If weak → ask user to re-enter.
Allow max 3 attempts.
If still weak → "Account Locked."

Password Strength + Re-entry Attempts

Code:

PROGRAM 15

[45]:
```
import re

attempts = 3
while attempts > 0:
    pwd = input("Enter Password: ")
    if (len(pwd) >= 8 and
        re.search("[A-Z]", pwd) and
        re.search("[a-z]", pwd) and
        re.search("[0-9]", pwd) and
        re.search("[@#$]", pwd)):
        print("Strong Password")
        break
    else:
        attempts -= 1
        print("Weak Password")
else:
    print("Account Locked")
```

Output:

```
Enter Password:  223344
Weak Password
Enter Password:  a2m345
Weak Password
Enter Password:  0A3e4@w
Weak Password
Account Locked
```

LAB NO 04

LAB EXERCISE

Program 16: Electricity Bill with Slabs & Penalty
Function calculate_bill(units, late_payment=False):
Apply slab rates (5/7/10 Rs).
If late_payment → add 15% penalty.
Return bill breakdown.

Electricity Bill

Code:

PROGRAM 16

```
[103]: def calculate_bill():
           try:
               units = int(input("Enter units consumed: "))
           except ValueError:
               print("Invalid input. Please enter a number for units.")
               return

           late = input("Late payment? (yes/no): ").strip().lower()

           if units <= 100:
               bill = units * 5
           elif units <= 300:
               bill = (100 * 5) + (units - 100) * 7
           else:
               bill = (100 * 5) + (200 * 7) + (units - 300) * 10

           if late == "yes":
               penalty = bill * 0.15
               bill += penalty
               print("Late Payment Penalty:", round(penalty, 2))
           elif late != "no":
               print("Invalid input for late payment. Please enter 'yes' or 'no'.")

           print("Total Electricity Bill:", round(bill, 2))
           return bill

       calculate_bill()
```

Output:

```
Enter units consumed:  305
Late payment? (yes/no):  no
Total Electricity Bill: 1950
[103]: 1950
```

Program 17: Cricket Match Predictor with Multiple Conditions
Function predict_score(runs, balls, wickets, overs_left):
Compute strike rate & run rate.
If run rate < 5 and wickets > 6 → "Losing."
If run rate > 7 and wickets < 5 → "Winning."
Else → "Balanced."

Cricket Match Predictor

Code:

PROGRAM 17

```
[97]: def predict_score(runs, balls, wickets, overs_left):
          # Prevent division by zero
          if balls == 0:
              return "Invalid input: balls cannot be zero"

          run_rate = runs / (balls / 6)

          if run_rate < 5 and wickets > 6:
              return "Losing"
          elif run_rate > 7 and wickets < 5:
              return "Winning"
          elif overs_left < 5 and wickets <= 2:
              return "Critical"
          else:
              return "Balanced"
      # Example Usage
      print(predict_score(120, 90, 7, 10))   # Balanced
      print(predict_score(250, 180, 3, 15))  # Winning
      print(predict_score(50, 60, 8, 20))    # Losing
      print(predict_score(180, 240, 2, 3))   # Critical
```

Output:

```
Balanced
Winning
Balanced
Critical
```

Program 18: . Hotel Booking with Tax & Discount
Function book_room(type, days, is_member):
Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.
If days > 7 → 20% discount.
If member → extra 10% discount.
Add 12% tax.
Return bill slip.

Hotel Booking System

Code:

PROGRAM 18

```
[93]: def book_room(room_type, days, is_member):
          rates = {"standard": 2000, "deluxe": 4000, "suite": 6000}

          if room_type not in rates:
              print("Invalid room type. Choose from:", ", ".join(rates.keys()))
              return None

          bill = rates[room_type] * days

          if days > 7:
              bill *= 0.8

          if is_member:
              bill *= 0.9

          bill *= 1.12

          return round(bill, 2)

      # Example usage:
      print("Standard, 5 days, non-member:", book_room("standard", 5, False))
      print("Deluxe, 10 days, member:", book_room("deluxe", 10, True))
      print("Invalid type:", book_room("luxury", 3, False))
```

Output:

```
Standard, 5 days, non-member: 11200.0
Deluxe, 10 days, member: 32256.0
Invalid room type. Choose from: standard, deluxe, suite
Invalid type: None
```

Program 19: . Flight Fare Calculator with International Tax
Function calculate_fare(distance, class_type, international=False):
Economy: 10/unit, Business: 25/unit, First: 40/unit.
Add fuel surcharge 500.
If international → add 18% tax.
If booking in advance (>30 days) → 10% discount.

Flight Fare Calculator

Code:

PROGRAM 19

[89]:
```python
def calculate_fare(distance, class_type, international=False):
    rates = {"economy": 10, "business": 25, "first": 40}

    if class_type not in rates:
        print("Invalid class type. Choose from:", ", ".join(rates.keys()))
        return None

    fare = distance * rates[class_type] + 500

    if international:
        fare *= 1.18

    return fare


# Example usage:
print("Economy Domestic:", calculate_fare(100, "economy"))
print("Business International:", calculate_fare(200, "business", international=True))
print("Invalid Class:", calculate_fare(50, "premium"))
```

Output:

```
Economy Domestic: 1500
Business International: 6490.0
Invalid class type. Choose from: economy, business, first
Invalid Class: None
```

Program 20: Smart Vending Machine with Wallet System
Function vending_machine(item, amount_paid, wallet_balance):
Items stored in dict with stock + price.
If sufficient → dispense + return change.

Smart Vending Machine

Code:

PROGRAM 20

[83]:
```python
def vending_machine():
    items = {
        "chips": {"price": 50, "stock": 5},
        "juice": {"price": 80, "stock": 0},
        "biscuits": {"price": 40, "stock": 3}
    }

    item = input("Select Item: ").lower()
    paid = int(input("Amount Paid: "))
    wallet = int(input("Wallet Balance: "))

    if item not in items:
        print("Item not available")
        return

    if items[item]["stock"] == 0:
        print("Out of stock.")
        for i in items:
            if items[i]["stock"] > 0:
                print("Try:", i)
        return

    price = items[item]["price"]

    if paid >= price:
        items[item]["stock"] -= 1
        change = paid - price
        wallet += change
        print("Item Dispensed:", item)
        print("Change Returned:", change)
        print("Updated Wallet Balance:", wallet)
    else:
        print("Insufficient Amount")
vending_machine()
```

Output:

```
Select Item:  chips
Amount Paid:  50
Wallet Balance:  100
Item Dispensed: chips
Change Returned: 0
Updated Wallet Balance: 100
```

LAB NO 05

OOP IN PYTHON

OOP in Python

- **Definition**: OOP is a programming paradigm where code is organized into classes and objects.
- **Objects**: Instances of classes that represent real-world entities.
- **Classes**: Blueprints for creating objects, defining attributes (data) and methods (functions).
- **Purpose**: To model complex systems by breaking them into smaller, reusable components.

Defining a Class and Object

Encapsulation

- Bundling data and methods together.
- **Example:** A Car class with attributes like speed and methods like drive().

Syntax:

```
class Car:
    def __init__(self, brand, speed):
        self.brand = brand      # attribute
        self.speed = speed

    def drive(self):            # method
        print(f"{self.brand} is driving at {self.speed} km/h")

# Create object
car1 = Car("Toyota", 120)
car1.drive()
```

Inheritance

- Allows a class to inherit properties/methods from another class.
- **Example:** Electric Car inherits from Car.

Syntax:

```python
class Car:
    def __init__(self, brand):
        self.brand = brand

    def drive(self):
        print(f"{self.brand} is driving")

class ElectricCar(Car):    # inherits from Car
    def charge(self):
        print(f"{self.brand} is charging")

tesla = ElectricCar("Tesla")
tesla.drive()
tesla.charge()
```

Polymorphism

- Same method name can behave differently depending on the object.
- **Example:** area () method in Circle vs Rectangle.

Syntax:

```python
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")
for pet in [Dog(), Cat()]:
    pet.speak()
```

Abstraction

- Hiding implementation details and exposing only necessary features.
- **Example:** You use. fit () in scikit-learn without knowing internal math.

Use of OOP in Python

Reusability: Classes can be reused across projects.

Modularity: Easier to debug and maintain.

Scalability: Suitable for large applications.

Real-world modeling: Mirrors how we think about entities (students, cars, customers).

LAB EXERCISE

Program 1: Create a `Bank Account` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

Code:

```
[109]: class BankAccount:
           def __init__(self, account_number, holder_name, balance=0.0):
               self.account_number = account_number
               self.holder_name = holder_name
               self.balance = float(balance)

           def deposit(self, amount):
               if amount <= 0:
                   print("Deposit amount must be positive.")
                   return
               self.balance += amount
               print(f"Deposited: {amount:.2f}")

           def withdraw(self, amount):
               if amount <= 0:
                   print("Withdrawal amount must be positive.")
                   return
               if amount > self.balance:
                   print("Withdrawal denied: insufficient funds.")
                   return
               self.balance -= amount
               print(f"Withdrawn: {amount:.2f}")

           def display_balance(self):
               print(f"Account {self.account_number} | Holder: {self.holder_name} | Balance: {self.balance:.2f}")
```

Output:

```
Account PK-001 | Holder: Tuf | Balance: 1000.00
Deposited: 500.00
Withdrawn: 1200.00
Withdrawal denied: insufficient funds.
Account PK-001 | Holder: Tuf | Balance: 300.00
```

Program 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

Code:

[111]:
```python
class Book:
    def __init__(self, title, author, isbn, available=True):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = available

    def __repr__(self):
        return f"{self.title} by {self.author} (ISBN: {self.isbn}) [{'Available' if self.available else 'Out'}]"


class Library:
    def __init__(self):
        self.catalog = {}  # isbn -> Book

    def add_book(self, book):
        self.catalog[book.isbn] = book
        print(f"Added: {book}")

    def search(self, *, title=None, author=None):
        results = []
        for b in self.catalog.values():
            if (title and title.lower() in b.title.lower()) or (author and author.lower() in b.author.lower()):
                results.append(b)
        return results
```

```python
    def checkout(self, isbn):
        b = self.catalog.get(isbn)
        if not b:
            print("Book not found.")
            return
        if not b.available:
            print("Book already checked out.")
            return
        b.available = False
        print(f"Checked out: {b.title}")

    def return_book(self, isbn):
        b = self.catalog.get(isbn)
        if not b:
            print("Book not found.")
            return
        b.available = True
        print(f"Returned: {b.title}")

    def available_books(self):
        return [b for b in self.catalog.values() if b.available]
```

Output:

```
Added: Clean Code by Robert C. Martin (ISBN: 9780132350884) [Available]
Added: Python Crash Course by Eric Matthes (ISBN: 9781593279288) [Available]
Search by author 'Eric': [Python Crash Course by Eric Matthes (ISBN: 9781593279288) [Available]]
Checked out: Clean Code
Available now: [Python Crash Course by Eric Matthes (ISBN: 9781593279288) [Available]]
Returned: Clean Code
Available now: [Clean Code by Robert C. Martin (ISBN: 9780132350884) [Available], Python Crash Course by Eric Matthes (ISBN: 9781593279288) [Available]]
```

Program 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

Code:

```
[113]: class Student:
           def __init__(self, student_id, name):
               self.student_id = student_id
               self.name = name
               self.grades = []

           def add_grade(self, score):
               if 0 <= score <= 100:
                   self.grades.append(score)
                   print(f"Added grade: {score}")
               else:
                   print("Invalid grade. Must be 0-100.")

           def average(self):
               return sum(self.grades) / len(self.grades) if self.grades else 0.0

           def letter_grade(self):
               avg = self.average()
               if avg >= 90: return "A"
               if avg >= 80: return "B"
               if avg >= 70: return "C"
               if avg >= 60: return "D"
               return "F"
```

```
    def summary(self):
        print(f"ID: {self.student_id} | Name: {self.name}")
        print(f"Grades: {self.grades}")
        print(f"Average: {self.average():.2f} | Letter: {self.letter_grade()}")
```

Output:

```
Added grade: 88
Added grade: 92
Added grade: 73
ID: FA-23-001 | Name: Ayesha
Grades: [88, 92, 73]
Average: 84.33 | Letter: B
```

Program 4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

Code:

[115]:
```python
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
        self.price = float(price)
        self.category = category

    def __repr__(self):
        return f"{self.name} ({self.category}) - Rs {self.price:.2f}"


class Order:
    TAX_RATE = 0.13  # 13%

    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)
        print(f"Added: {item}")

    def subtotal(self):
        return sum(i.price for i in self.items)

    def apply_discount(self, percent):
        sub = self.subtotal()
        discount = sub * (percent / 100.0)
        return max(sub - discount, 0.0)
```

```python
def receipt(self, discount_percent=0):
    print("---- Receipt ----")
    for i in self.items:
        print(f"{i.name}: Rs {i.price:.2f}")
    sub = self.subtotal()
    print(f"Subtotal: Rs {sub:.2f}")
    if discount_percent:
        after_disc = self.apply_discount(discount_percent)
        print(f"Discount ({discount_percent}%): -Rs {(sub - after_disc):.2f}")
        total = self.total_with_tax(after_disc)
    else:
        total = self.total_with_tax()
    print(f"Tax ({int(self.TAX_RATE*100)}%): Rs {(total - (sub if not discount_percent else self.apply_discount(discount_percent))):.2f}")
    print(f"Total: Rs {total:.2f}")
    print("-----------------")
```

Output:

```
Added: Burger (Food) - Rs 500.00
Added: Tea (Beverage) - Rs 100.00
---- Receipt ----
Burger: Rs 500.00
Tea: Rs 100.00
Subtotal: Rs 600.00
Discount (10%): -Rs 60.00
Tax (13%): Rs 70.20
Total: Rs 610.20
-----------------
```

Program 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

Code:

```
[117]: class Vehicle:
           def __init__(self, license_plate, model, price_per_day, available=True):
               self.license_plate = license_plate
               self.model = model
               self.price_per_day = float(price_per_day)
               self.available = available

           def __repr__(self):
               return f"{self.model} [{self.license_plate}] - Rs {self.price_per_day:.2f}/day ({'Available' if self.available else 'Rented'})"


       class Car(Vehicle): pass
       class Motorcycle(Vehicle): pass
       class Truck(Vehicle): pass


       class RentalService:
           def __init__(self):
               self.fleet = {}

           def add_vehicle(self, v):
               self.fleet[v.license_plate] = v
               print(f"Added: {v}")

           def rent(self, license_plate, days):
               v = self.fleet.get(license_plate)
               if not v:
                   print("Vehicle not found.")
                   return None
               if not v.available:
                   print("Vehicle not available.")
                   return None
```

```
            v.available = False
            cost = v.price_per_day * days
            print(f"Rented {v.model} for {days} days. Cost: Rs {cost:.2f}")
            return cost

        def return_vehicle(self, license_plate):
            v = self.fleet.get(license_plate)
            if v:
                v.available = True
                print(f"Returned: {v.model}")
```

Output:

```
Added: Honda Civic [LEH-123] - Rs 5000.00/day (Available)
Added: Yamaha YBR [FSD-456] - Rs 1500.00/day (Available)
Added: Hino 300 [ISB-789] - Rs 8000.00/day (Available)
Rented Honda Civic for 3 days. Cost: Rs 15000.00
Fleet: [Honda Civic [LEH-123] - Rs 5000.00/day (Rented), Yamaha YBR [FSD-456] - Rs 1500.00/day (Available), Hino 300 [ISB-789] - Rs 8000.00/day (Availab
e)]
Returned: Honda Civic
Fleet: [Honda Civic [LEH-123] - Rs 5000.00/day (Available), Yamaha YBR [FSD-456] - Rs 1500.00/day (Available), Hino 300 [ISB-789] - Rs 8000.00/day (Avai
able)]
```

LAB NO 06

LAB EXERCISE

Program 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

Code:

```
[119]: from datetime import date

class Employee:
    def __init__(self, emp_id, name, position, monthly_salary, start_date):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.monthly_salary = float(monthly_salary)
        self.start_date = start_date  # datetime.date

    def annual_salary(self):
        return self.monthly_salary * 12

    def apply_raise(self, percent):
        self.monthly_salary *= (1 + percent / 100.0)
        print(f"New monthly salary: Rs {self.monthly_salary:.2f}")

    def employment_duration_months(self):
        today = date.today()
        months = (today.year - self.start_date.year) * 12 + (today.month - self.start_date.month)
        return max(months, 0)

    def details(self):
        print(f"{self.emp_id}: {self.name} ({self.position})")
        print(f"Monthly: Rs {self.monthly_salary:.2f} | Annual: Rs {self.annual_salary():.2f}")
        print(f"Employed for ~{self.employment_duration_months()} months")
```

Output:

```
EMP-007: Amna (Engineer)
Monthly: Rs 120000.00 | Annual: Rs 1440000.00
Employed for ~31 months
New monthly salary: Rs 132000.00
EMP-007: Amna (Engineer)
Monthly: Rs 132000.00 | Annual: Rs 1584000.00
Employed for ~31 months
```

Program 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

Code:

[123]:
```python
class Product:
    def __init__(self, pid, name, price, stock):
        self.pid = pid
        self.name = name
        self.price = float(price)
        self.stock = int(stock)

    def __repr__(self):
        return f"{self.name} (ID: {self.pid}) - Rs {self.price:.2f} | Stock: {self.stock}"


class ShoppingCart:
    def __init__(self):
        self.items = {}  # pid -> quantity

    def add_product(self, product, qty=1):
        if product.stock < qty:
            print("Not enough stock.")
            return
        self.items[product.pid] = self.items.get(product.pid, 0) + qty
        product.stock -= qty
        print(f"Added {qty} x {product.name}")
```

```python
    def remove_product(self, product, qty=1):
        current = self.items.get(product.pid, 0)
        if current < qty:
            print("Cannot remove more than in cart.")
            return
        self.items[product.pid] = current - qty
        product.stock += qty
        if self.items[product.pid] == 0:
            del self.items[product.pid]
        print(f"Removed {qty} x {product.name}")

    def total(self, inventory):
        total = 0.0
        for pid, qty in self.items.items():
            total += inventory[pid].price * qty
        return total

    def checkout(self, inventory):
        total_cost = self.total(inventory)
        print(f"Checkout total: Rs {total_cost:.2f}")
        self.items.clear()
        print("Cart cleared.")
```

Output:

```
Laptop (ID: 1) - Rs 150000.00 | Stock: 5
Added 2 x Laptop
Added 3 x Mouse
Inventory after add: Laptop (ID: 1) - Rs 150000.00 | Stock: 3 Mouse (ID: 2) - Rs 1500.00 | Stock: 17
Removed 1 x Mouse
Total: 303000.0
Checkout total: Rs 303000.00
Cart cleared.
Inventory after checkout: Laptop (ID: 1) - Rs 150000.00 | Stock: 3 Mouse (ID: 2) - Rs 1500.00 | Stock: 18
```

Program 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

Code:

```
[127]: from datetime import datetime

class Patient:
    def __init__(self, pid, name, age, history=None):
        self.pid = pid
        self.name = name
        self.age = age
        self.history = history or []

    def add_history(self, record):
        self.history.append(record)

    def __repr__(self):
        return f"{self.pid} - {self.name}, {self.age} yrs"


class Appointment:
    def __init__(self):
        self.bookings = []  # list of dicts

    def schedule(self, patient, doctor, when):
        self.bookings.append({"patient": patient, "doctor": doctor, "datetime": when})
        print(f"Scheduled: {patient.name} with Dr.{doctor} at {when}")
```

```
    def cancel(self, patient, when):
        for b in self.bookings:
            if b["patient"] == patient and b["datetime"] == when:
                self.bookings.remove(b)
                print("Appointment cancelled.")
                return
        print("Appointment not found.")

    def view(self):
        for b in self.bookings:
            print(f"{b['datetime']} | {b['patient'].name} -> Dr.{b['doctor']}")
```

Output:

```
Scheduled: Aamu with Dr.Sara at 2026-01-05 10:00:00
2026-01-05 10:00:00 | Aamu -> Dr.Sara
Appointment cancelled.
```

Program 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

Code:

[131]:
```python
class Post:
    def __init__(self, content, privacy="public"):
        self.content = content
        self.privacy = privacy  # "public" or "friends"

    def __repr__(self):
        return f"[{self.privacy}] {self.content}"


class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = set()
        self.posts = []

    def add_friend(self, other):
        self.friends.add(other.username)
        other.friends.add(self.username)
        print(f"{self.username} and {other.username} are now friends.")

    def create_post(self, content, privacy="public"):
        p = Post(content, privacy)
        self.posts.append(p)
        print(f"Post created: {p}")

    def timeline(self, viewer_username=None):
        visible = []
        for p in self.posts:
            if p.privacy == "public":
                visible.append(p)
            elif p.privacy == "friends" and viewer_username in self.friends:
                visible.append(p)
        print(f"Timeline for viewer '{viewer_username}': {visible}")
```

Output:

```
Post created: [public] Hello world!
Post created: [friends] Private to friends
Timeline for viewer 'None': [[public] Hello world!]
tuf and hira are now friends.
Timeline for viewer 'hira': [[public] Hello world!, [friends] Private to friends]
```

LAB NO 07

LAB EXERCISE

Program 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

Code:

```
[133]: from datetime import date

class Room:
    def __init__(self, number, rtype, price, available=True):
        self.number = number
        self.rtype = rtype
        self.price = float(price)
        self.available = available

    def __repr__(self):
        return f"Room {self.number} ({self.rtype}) - Rs {self.price:.2f} [{ 'Available' if self.available else 'Booked' } ]"


class Booking:
    def __init__(self, room, check_in, check_out):
        if not room.available:
            raise ValueError("Room not available.")
        self.room = room
        self.check_in = check_in
        self.check_out = check_out
        self.room.available = False

    def days(self):
        return (self.check_out - self.check_in).days

    def total_cost(self):
        return self.days() * self.room.price

    def checkout(self):
        self.room.available = True
        print(f"Checked out room {self.room.number}. Total: Rs {self.total_cost():.2f}")
```

Output:

```
Room 101 (double) - Rs 8000.00 [Available ]
Stay days: 3
Total cost: 24000.0
Checked out room 101. Total: Rs 24000.00
Room 101 (double) - Rs 8000.00 [Available ]
```

Program 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

Code:

[135]:
```python
class InvProduct:
    def __init__(self, pid, name, qty, price, supplier):
        self.pid = pid
        self.name = name
        self.qty = int(qty)
        self.price = float(price)
        self.supplier = supplier

    def __repr__(self):
        return f"{self.name} ({self.pid}) qty:{self.qty} Rs{self.price:.2f} supplier:{self.supplier}"


class Inventory:
    def __init__(self, low_stock_threshold=5):
        self.items = {}  # pid -> InvProduct
        self.low_stock_threshold = low_stock_threshold

    def add_product(self, p):
        self.items[p.pid] = p
        print(f"Added: {p}")

    def update_qty(self, pid, delta):
        p = self.items.get(pid)
        if not p:
            print("Product not found.")
            return
        p.qty = max(p.qty + delta, 0)
        print(f"Updated qty: {p}")
        if p.qty <= self.low_stock_threshold:
            print(f"Alert: Low stock for {p.name} (qty {p.qty})")

    def low_stock(self):
        return [p for p in self.items.values() if p.qty <= self.low_stock_threshold]
```

Output:

```
Added: Screws (1) qty:10 Rs2.50 supplier:ABC Supplies
Added: Nails (2) qty:3 Rs1.00 supplier:XYZ Co
Updated qty: Screws (1) qty:2 Rs2.50 supplier:ABC Supplies
Alert: Low stock for Screws (qty 2)
Low stock: [Screws (1) qty:2 Rs2.50 supplier:ABC Supplies, Nails (2) qty:3 Rs1.00 supplier:XYZ Co]
Updated qty: Nails (2) qty:1 Rs1.00 supplier:XYZ Co
Alert: Low stock for Nails (qty 1)
```

Program 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

Code:

[137]:
```python
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title = title
        self.genre = genre
        self.duration = duration
        self.showtimes = showtimes

    def __repr__(self):
        return f"{self.title} ({self.genre}) {self.duration} mins {self.showtimes}"


class Theater:
    def __init__(self, rows=3, cols=4):
        self.seats = [[True for _ in range(cols)] for _ in range(rows)]  # True=available

    def reserve(self, row, col, category="standard"):
        if not self.seats[row][col]:
            print("Seat already booked.")
            return None
        self.seats[row][col] = False
        price = 800 if category == "standard" else 1200
        print(f"Reserved seat ({row},{col}) [{category}] Price: Rs {price}")
        return price

    def available_count(self):
        return sum(seat for row in self.seats for seat in row)
```

Output:

```
Inception (Sci-Fi) 148 mins ['6 PM', '9 PM']
Reserved seat (0,1) [standard] Price: Rs 800
Seat already booked.
Reserved seat (1,2) [premium] Price: Rs 1200
Available seats: 4
```

Program 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

Code:

[139]:
```python
class Member:
    def __init__(self, mid, name, mtype):
        self.mid = mid
        self.name = name
        self.mtype = mtype  # basic/premium
        self.attendance = 0

    def check_in(self):
        self.attendance += 1
        print(f"{self.name} checked in. Total attendance: {self.attendance}")


class Trainer:
    def __init__(self, tid, name, specialty):
        self.tid = tid
        self.name = name
        self.specialty = specialty

    def __repr__(self):
        return f"{self.name} ({self.specialty})"


class SessionScheduler:
    def __init__(self):
        self.sessions = []  # list of tuples (member, trainer, time)

    def schedule(self, member, trainer, time):
        self.sessions.append((member, trainer, time))
        print(f"Scheduled: {member.name} with {trainer} at {time}")

    def view(self):
        for m, t, tm in self.sessions:
            print(f"{tm} - {m.name} with {t.name} ({t.specialty})")
```

Output:

```
Sana checked in. Total attendance: 1
Scheduled: Sana with Usman (Strength) at Tue 6 PM
Tue 6 PM - Sana with Usman (Strength)
```

Program 15: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

Code:

[141]:
```python
class BankAccountBase:
    def __init__(self, acc_no, holder, balance=0.0):
        self.acc_no = acc_no
        self.holder = holder
        self.balance = float(balance)

    def deposit(self, amt):
        if amt > 0:
            self.balance += amt
            print(f"Deposited {amt:.2f} into {self.acc_no}")

    def withdraw(self, amt):
        raise NotImplementedError

    def transfer_to(self, other, amt):
        if self.withdraw(amt):
            other.deposit(amt)
            print(f"Transferred {amt:.2f} from {self.acc_no} to {other.acc_no}")


class SavingsAccount(BankAccountBase):
    INTEREST_RATE = 0.04

    def apply_interest(self):
        interest = self.balance * self.INTEREST_RATE
        self.balance += interest
        print(f"Interest applied: {interest:.2f}")
```

```python
    def withdraw(self, amt):
        if amt <= 0 or amt > self.balance:
            print("Savings withdrawal denied.")
            return False
        self.balance -= amt
        print(f"Savings withdrawn: {amt:.2f}")
        return True


class CheckingAccount(BankAccountBase):
    OVERDRAFT_LIMIT = 5000

    def withdraw(self, amt):
        if amt <= 0:
            print("Invalid amount.")
            return False
        if amt > self.balance + self.OVERDRAFT_LIMIT:
            print("Checking withdrawal denied: exceeds overdraft.")
            return False
        self.balance -= amt
        print(f"Checking withdrawn: {amt:.2f}")
        return True
```

Output:

```
Interest applied: 800.00
Savings withdrawn: 3000.00
Deposited 3000.00 into CA-1
Transferred 3000.00 from SA-1 to CA-1
Checking withdrawn: 9000.00
Checking withdrawn: 2000.00
Savings balance: 17800.0
Checking balance: -3000.0
```

LAB NO 08

LAB EXERCISE

Program 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

Code:

```
[143]: class Pizza:
           SIZE_PRICES = {"small": 800, "medium": 1200, "large": 1600}
           CRUST_PRICES = {"thin": 0, "regular": 100, "cheese_stuffed": 300}
           TOPPING_PRICE = 120

           def __init__(self, size, crust, toppings=None):
               self.size = size
               self.crust = crust
               self.toppings = toppings or []

           def price(self):
               base = self.SIZE_PRICES[self.size] + self.CRUST_PRICES[self.crust]
               return base + len(self.toppings) * self.TOPPING_PRICE

           def __repr__(self):
               return f"{self.size} {self.crust} pizza with {self.toppings} -> Rs {self.price()}"
```

Output:

```
medium regular pizza with ['pepperoni', 'olives'] -> Rs 1540
large cheese_stuffed pizza with ['mushroom', 'corn', 'jalapeno'] -> Rs 2260
```

Program 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

Code:

[145]:

```python
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = float(price)
        self.passed_qc = False

    def quality_check(self):
        self.passed_qc = self.year >= 2020 and self.price > 0
        return self.passed_qc

    def __repr__(self):
        return f"{self.year} {self.model} ({self.color}) Rs {self.price:.2f} QC:{self.passed_qc}"


class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.production_log = []

    def produce(self, model, year, color, price):
        car = Car(model, year, color, price)
        qc = car.quality_check()
        status = "PASSED" if qc else "FAILED"
        self.production_log.append(car)
        print(f"Produced: {car} | QC {status}")
        return car

    def report(self):
        print(f"Manufacturer: {self.name}")
        for c in self.production_log:
            print(c)
```

Output:

```
Produced: 2022 Alto (white) Rs 2100000.00 QC:True | QC PASSED
Produced: 2015 Classic (blue) Rs 1500000.00 QC:False | QC FAILED
Manufacturer: Pak Motors
2022 Alto (white) Rs 2100000.00 QC:True
2015 Classic (blue) Rs 1500000.00 QC:False
```

Program 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

Code:

[147]:

```
class Teacher:
    def __init__(self, name):
        self.name = name


class Student:
    def __init__(self, sid, name):
        self.sid = sid
        self.name = name
        self.grades = {}
        self.attendance = 0


class Classroom:
    def __init__(self, teacher):
        self.teacher = teacher
        self.students = {}

    def add_student(self, s):
        self.students[s.sid] = s
        print(f"Added student: {s.name}")

    def mark_attendance(self, sid):
        s = self.students.get(sid)
        if s:
            s.attendance += 1
            print(f"{s.name} attendance: {s.attendance}")
```

```
    def grade_assignment(self, sid, assignment, score):
        s = self.students.get(sid)
        if s:
            s.grades[assignment] = score
            print(f"Graded {s.name}: {assignment} = {score}")

    def report(self):
        print(f"Teacher: {self.teacher.name}")
        for s in self.students.values():
            avg = sum(s.grades.values()) / len(s.grades) if s.grades else 0
            print(f"{s.sid} {s.name} | Attendance: {s.attendance} | Avg: {avg:.2f}")
```

Output:

```
Added student: Amna
Added student: Minha
Amna attendance: 1
Graded Amna: HW1 = 85
Graded Amna: Quiz1 = 78
Minha attendance: 1
Graded Minha: HW1 = 92
Teacher: Sir Bilal
ST-1 Amna | Attendance: 1 | Avg: 81.50
ST-2 Minha | Attendance: 1 | Avg: 92.00
```

Program19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

Code:

[151]:

```
class Flight:
    def __init__(self, number, destination, departure_time, capacity):
        self.number = number
        self.destination = destination
        self.departure_time = departure_time
        self.capacity = capacity
        self.passengers = []

    def reserve_seat(self, passenger):
        if len(self.passengers) >= self.capacity:
            print("Flight full.")
            return False
        self.passengers.append(passenger)
        print(f"Seat reserved for {passenger.name}")
        return True

    def manifest(self):
        print(f"Manifest for {self.number} -> {self.destination}")
        for p in self.passengers:
            print(f"- {p.name} ({p.pid})")


class Passenger:
    def __init__(self, pid, name):
        self.pid = pid
        self.name = name
```

Output:

```
Seat reserved for Hamza
Seat reserved for Iqra
Flight full.
Manifest for PK-309 -> Karachi
- Hamza (PS-1)
- Iqra (PS-2)
```

Program 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

Code:

[153]:

```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "off"

    def turn_on(self):
        self.status = "on"
        print(f"{self.name} turned on.")

    def turn_off(self):
        self.status = "off"
        print(f"{self.name} turned off.")


class SmartLight(SmartDevice):
    def set_brightness(self, level):
        print(f"{self.name} brightness set to {level}%")


class Thermostat(SmartDevice):
    def set_temperature(self, temp):
        print(f"{self.name} temperature set to {temp}°C")


class SecurityCamera(SmartDevice):
    def start_recording(self):
        print(f"{self.name} started recording.")
```

```
class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, d):
        self.devices.append(d)
        print(f"Added device: {d.name}")

    def routine_night(self):
        print("Running 'Night' routine...")
        for d in self.devices:
            if isinstance(d, SmartLight):
                d.turn_off()
            elif isinstance(d, Thermostat):
                d.set_temperature(22)
            elif isinstance(d, SecurityCamera):
                d.turn_on()
                d.start_recording()
```

Output:

```
Added device: Living Room Light
Added device: Main Thermostat
Added device: Front Camera
Living Room Light turned on.
Running 'Night' routine...
Living Room Light turned off.
Main Thermostat temperature set to 22°C
Front Camera turned on.
Front Camera started recording.
```

LAB NO 09

INTRODUCTION TO NUMPY

NumPy

- **Definition**: NumPy stands for *Numerical Python*. It provides an N-dimensional array object (nd array) and a collection of mathematical functions to operate on these arrays.
- **Purpose**: To perform fast numerical computations on large datasets.
- **Performance**: NumPy arrays are stored in contiguous memory, making them up to **50x faster** than Python lists.

Key Features

- **N-dimensional arrays (ndarray)**: Core data structure for homogeneous data types.
- **Broadcasting**: Allows operations on arrays of different shapes without explicit looping.
- **Mathematical functions**: Linear algebra, Fourier transforms, statistics, etc.
- **Integration**: Works seamlessly with libraries like Pandas, SciPy, and scikit-learn.

Basic Syntax

- **import numpy as np**

Creating Arrays

```
arr = np.array([1, 2, 3, 4, 5])

print(arr)   # Output: [1 2 3 4 5]
```

Array Operations

```
arr = np.array([10, 20, 30])

print(arr + 5)     # [15 25 35]

print(arr * 2)     # [20 40 60]
```

Multi-dimensional Arrays

```
matrix = np.array([[1, 2], [3, 4]])

print(matrix)
```

Output:

```
[[1 2]
 [3 4]]
```

Mathematical Operations

```
arr = np.array([1, 2, 3])
print(np.mean(arr))   # Average → 2.0
print(np.std(arr))    # Standard deviation → 0.816
print(np.sum(arr))    # Sum → 6
```

Comparison: Python Lists vs NumPy Arrays

Feature	Python List	NumPy Array
Speed	Slower	Much faster (C-based)
Memory efficiency	Higher usage	Lower usage
Mathematical ops	Manual loops	Vectorized operations
Broadcasting	Not supported	Supported

Challenges & Considerations

- **Homogeneous data**: NumPy arrays require elements of the same type.
- **Learning curve**: Broadcasting rules can be tricky for beginners.
- **Dependency**: For advanced tasks, NumPy is often used with Pandas or SciPy.

LAB EXERCISE

Program 1: A weather station records temperatures (°C) for 7 days:
[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- **Convert the list into a NumPy array**
- **Find:**
    - **Average temperature**
    - **Maximum and minimum temperature**
    - **Temperature difference (range)**

Code:

PROGRAM 1

```
[4]: import numpy as np

[5]: temp = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])

[6]: temp_avg = np.mean(temp)

[7]: print(temp_avg)
     23.12857142857143

[8]: max_temp = np.max(temp)
     min_temp = np.min(temp)

[9]: print (max_temp)
     print (min_temp)
     26.5
     19.8

[10]: temp_diff = np.max(temp) - np.min(temp)

[11]: print(temp_diff)
      6.699999999999999
```

Program 2: A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- **Create a 4×7 NumPy array**
- **Find weekly totals**
- **Find the highest sale day (index only)**
- **Find the week with the most sales**

Code:

PROGRAM 2

```
[13]: sales = np.array([
          [12, 15, 14, 10, 9, 8, 7],
           [11, 12, 13, 9, 5, 8, 6],
           [7, 9, 12, 10, 11, 13, 12],
           [10, 11, 12, 7, 8, 9, 11]
      ])

[14]: print("4*7 Numpy Array\n",sales)

      4*7 Numpy Array
       [[12 15 14 10  9  8  7]
       [11 12 13  9  5  8  6]
       [ 7  9 12 10 11 13 12]
       [10 11 12  7  8  9 11]]

[15]: weekly_totals = np.sum(sales, axis = 1)

[16]: print (weekly_totals)

      [75 64 74 68]

[17]: highest_sale = np.argmax(sales)

[18]: print ("Index of highest sale day = ", highest_sale)

      Index of highest sale day =  1

[19]: max_sales = np.argmax(sales)

[20]: print ("Week with the most sales is:",max_sales)

      Week with the most sales is: 1
```

Program 3: Students received marks in a test:
[56, 78, 45, 90, 88, 76, 59, 62]

Task:

- **Convert to NumPy array**
- **Add 5 bonus marks to every student using broadcasting**
- **Count how many students now scored > 60**
- **Find the median and standard deviation**

Code:

PROGRAM 3

```
[22]: test_marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])

[23]: print("Numpy Array:",test_marks)
      Numpy Array: [56 78 45 90 88 76 59 62]

[24]: uploaded_marks = test_marks + 5

[25]: print ("Bonus marks using broadcasting:", uploaded_marks)
      Bonus marks using broadcasting: [61 83 50 95 93 81 64 67]

[26]: score = np.sum(uploaded_marks > 60)

[27]: print ("Students who scored marks above 60:",score)
      Students who scored marks above 60: 7

[28]: median = np.median(uploaded_marks)

[29]: print ("Median:",median)
      Median: 74.0

[30]: std_dev = np.std(uploaded_marks)

[31]: print("Standard Deviation is:", std_dev)
      Standard Deviation is: 15.105876340020794
```

Program 4: A smartwatch stores daily step-counts for a user for 30 days.

Task:

- **Generate a random NumPy array of shape (30,) with values between 2000–15000**
- **Reshape into a 5×6 matrix**
- **Find:**
    - **Weekly average steps**
    - **Day with maximum steps**
    - **Number of days user crossed 10,000 steps**

Code:

PROGRAM 4

```
[33]: steps = np.random.randint(2000, 15000, size=30)

[34]: print (steps)

[ 6198 11402  5526  3882  9864  6244 10259 14920  5792  4240  9057  8362
  5743 12615  2033  3384 10692 13681  5899 11161  6399 13174  7532  7856
  6860  7438  9401  9407 13944 10701]

[35]: array_reshape = steps.reshape(5,6)

[36]: print(array_reshape)

[[ 6198 11402  5526  3882  9864  6244]
 [10259 14920  5792  4240  9057  8362]
 [ 5743 12615  2033  3384 10692 13681]
 [ 5899 11161  6399 13174  7532  7856]
 [ 6860  7438  9401  9407 13944 10701]]

[37]: avg_steps = np.mean(array_reshape, axis =1)

[38]: print ("Weekly average steps:\n", avg_steps)

Weekly average steps:
 [7186.         8771.66666667 8024.66666667 8670.16666667 9625.16666667]

[39]: max_index = np.argmax(steps)

[40]: print (max_index)

7

[41]: days_over_10000 = np.sum(steps > 10000)

[42]: print ("Number of days user crossed 10k:", days_over_10000)

Number of days user crossed 10k: 10
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- **Use array shape = (10, 4)**

Task:

- **Create an integer array using np.random.randint(60,150,(10,4))**
- **For each patient:**
  - **Compute daily average**
  - **Identify abnormal readings > 120**
- **For all patients:**
  - **Compute overall maximum and minimum**

Code:

PROGRAM 5

[82]:
```
heart_rate = np.random.randint(60,150,(10,4))
```

[86]:
```
print ("Heart Rate Data (10 patients × 4 readings):")
print (heart_rate)
```

```
Heart Rate Data (10 patients × 4 readings):
[[117 120 143  63]
 [ 83  77  79 129]
 [ 77 115  64 133]
 [ 97 115  62  78]
 [ 66  63 107  60]
 [ 78 135  67  61]
 [100 110 112 134]
 [145 119  65 121]
 [ 62  83 137 133]
 [ 61  74 146  63]]
```

[88]:
```
daily_avg = np.mean(heart_rate, axis = 1)
```

[90]:
```
print ("Daily Average Per Patient:", daily_avg)
```

```
Daily Average Per Patient: [110.75  92.    97.25  88.    74.    85.25 114.   112.5  103.75  86.  ]
```

[92]:
```
ab_reads = heart_rate > 120
```

[94]:
```
print (" Abnormal Readings Are: ", ab_reads)
```

```
Abnormal Readings Are:  [[False False  True False]
[False False False  True]
[False False False  True]
[False False False False]
[False False False False]
[False  True False False]
[False False False  True]
[ True False False  True]
[False False  True  True]
[False False  True False]]
```

Output:

[96]:
```
overall_max = np.max(heart_rate)
```

[102]:
```
print ("Overall Maximum Heart Rate For All Patients:",overall_max)
```

```
Overall Maximum Heart Rate For All Patients: 146
```

[104]:
```
overall_min = np.min(heart_rate)
```

[106]:
```
print ("Overall Minimum Heart Rate For All Patients:",overall_min)
```

```
Overall Minimum Heart Rate For All Patients: 60
```

LAB NO 10

LAB EXERCISE

Question 6: Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- **Compute difference between branches**
- **Calculate:**
    - **Average monthly sales of each branch**
    - **Which branch performed better overall?**
- **Compute correlation between A and B**

Code:

PROGRAM 6

```
[109]: # Data
A = np.array([122, 135, 150, 160, 155, 140])
B = np.array([110, 130, 148, 165, 158, 145])

[111]: difference = A - B

[113]: print("Difference between A and B (month-wise):")
print(difference)

Difference between A and B (month-wise):
[12  5  2 -5 -3 -5]

[115]: avg_A = np.mean(A)
avg_B = np.mean(B)

[117]: print("Average of Branch A:", avg_A)
print("Average of Branch B:", avg_B)

Average of Branch A: 143.66666666666666
Average of Branch B: 142.66666666666666

[119]: if avg_A > avg_B:
    print("\nBranch A performed better overall.")
elif avg_B > avg_A:
    print("\nBranch B performed better overall.")
else:
    print("\nBoth branches performed equally.")

Branch A performed better overall.
```

Output:

```
[121]: # Correlation between A and B
corr = np.corrcoef(A, B)[0, 1]
print("\nCorrelation between A and B:", corr)

Correlation between A and B: 0.9811677353198398
```

Question 7: Given a 4×4 grayscale image:

[[100, 120, 130, 90],

[80, 110, 140, 100],

[70, 90, 120, 130],

[110, 140, 150, 160]]

Task:

- **Convert to NumPy array**
- **Normalize pixel values (0–1)**
- **Apply a transformation: new_pixel = pixel * 1.2**
- **Clip all values > 255**
- **Compute average brightness before & after transformation**

Code:

PROGRAM 7

[124]:
```
img = np.array([
    [100, 120, 130,  90],
    [ 80, 110, 140, 100],
    [ 70,  90, 120, 130],
    [110, 140, 150, 160]
])
```

[126]:
```
print("Original Image:\n", img)
```

```
Original Image:
 [[100 120 130  90]
 [ 80 110 140 100]
 [ 70  90 120 130]
 [110 140 150 160]]
```

[128]:
```
# Normalize pixel values (0-1)
normalized = img / 255
print("\nNormalized Image:\n", normalized)
```

```
Normalized Image:
 [[0.39215686 0.47058824 0.50980392 0.35294118]
 [0.31372549 0.43137255 0.54901961 0.39215686]
 [0.2745098  0.35294118 0.47058824 0.50980392]
 [0.43137255 0.54901961 0.58823529 0.62745098]]
```

```
[130]: transformed = img * 1.2

[132]: # Clip values > 255
       transformed_clipped = np.clip(transformed, 0, 255)
       print("\nTransformed (Clipped) Image:\n", transformed_clipped)

       Transformed (Clipped) Image:
        [[120. 144. 156. 108.]
        [ 96. 132. 168. 120.]
        [ 84. 108. 144. 156.]
        [132. 168. 180. 192.]]

[134]: # Average brightness before & after
       avg_before = np.mean(img)
       avg_after = np.mean(transformed_clipped)
```

Output:

```
[136]: print("\nAverage brightness before:", avg_before)
       print("Average brightness after:", avg_after)

       Average brightness before: 115.0
       Average brightness after: 138.0
```

Question 8: Dataset contains 100 samples with 4 features each.

Task:

- **Generate a 100×4 matrix with values 1–100**
- **Standardize the dataset using:**
- **x_scaled = (x - mean) / std**
- **Perform:**
    - **Column-wise mean (should be ≈ 0)**
    - **Column-wise variance (should be ≈ 1)**

Verify results using NumPy functions

Code:

PROGRAM 8

[139]:
```
# Generate dataset
data = np.random.randint(1, 101, (100, 4))
print("Original Data:\n", data)
```

```
Original Data:
 [[ 91  74  52  44]
 [ 73  14  13  41]
 [ 93  71  52  69]
 [ 56   8  63  72]
 [ 97  39  73  90]
 [ 24  73  84   3]
 [ 10  66  66  54]
 [ 60  80  52  12]
 [ 62  48  63  12]
 [ 54   9  42  82]
 [ 16  46  79  77]
 [ 18  45  76  38]
 [ 69  82  58  87]
 [ 19  47  72  95]
 [ 81  33  96  11]
 [  5  50  54  60]
 [ 28  97  31  68]
```

[141]:
```
# Standardization
mean = np.mean(data, axis=0)
std = np.std(data, axis=0)

x_scaled = (data - mean) / std
```

[143]:
```
# Column-wise mean (after scaling)
scaled_mean = np.mean(x_scaled, axis=0)
```

[145]:
```
# Column-wise variance (after scaling)
scaled_var = np.var(x_scaled, axis=0)
```

Output:

```
[147]: # Output
print("\nOriginal Mean:", mean)
print("Original Std:", std)

print("\nScaled Data (first 5 rows):\n", x_scaled[:5])
```

```
Original Mean: [50.61 51.17 57.91 48.72]
Original Std: [29.32878961 26.73314609 26.76643234 29.5699442 ]

Scaled Data (first 5 rows):
 [[ 1.37714514  0.85399601 -0.22079894 -0.15962154]
 [ 0.76341371 -1.39040874 -1.67784781 -0.26107591]
 [ 1.44533752  0.74177577 -0.22079894  0.68583153]
 [ 0.18377847 -1.61484922  0.19016356  0.7872859 ]
 [ 1.58172228 -0.4552401   0.56376583  1.3960121 ]]
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- **Generate array shape (24, 6)**
- **Find hourly average**
- **Identify the busiest sensor (highest total density)**
- **Apply a threshold:**
    - **Replace all values above 50 with 50 (traffic cap)**
- **Compute energy usage:**

energy = Σ(sensor density × 0.5)

Code:

PROGRAM 9

[150]:
```
# Generate traffic density data (24 hours × 6 sensors)
data = np.random.randint(0, 100, (24, 6))
print("Traffic Density Data:\n", data)
```

```
Traffic Density Data:
 [[19 38 85 18 92 32]
 [64 34  6 69 90  5]
 [38 81 65 32 40 83]
 [83 36 81 83 79 98]
 [88 90 82 90 84 72]
 [23 66 24 61 68 85]
 [86 10 53 24  2 95]
 [93 70 81 42 40 57]
 [47  3 20  0 48 50]
 [95 79 56 85 30 83]
 [11 46 84 41 51 51]
 [82 10 57  9 60  4]
 [42  3 42 79 48 18]
 [18 12 32 33 75  2]
 [47 76 22 70 22 88]
 [51 60 75 22 53 34]
 [74 40 15 63 85 37]
 [48  4 92 29 64 66]
 [48 66  9 60 61 48]
 [65 78 27 70 94 28]
 [57 72 29 25 61  6]
 [98 91 14 80  5 98]
 [17 52 26 56 81 59]
 [91 86 72 99 43 81]]
```

[152]:
```
# Hourly average (mean across sensors)
hourly_avg = np.mean(data, axis=1)
print("\nHourly Average Traffic:\n", hourly_avg)
```

```
Hourly Average Traffic:
 [47.33333333 44.66666667 56.5        76.66666667 84.33333333 54.5
 45.         63.83333333 28.         71.33333333 47.33333333 37.
 38.66666667 28.66666667 54.16666667 49.16666667 52.33333333 50.5
 48.66666667 60.33333333 41.66666667 64.33333333 48.5        78.66666667]
```

[154]:
```
# Busiest sensor (highest total traffic)
total_per_sensor = np.sum(data, axis=0)
busiest_sensor = np.argmax(total_per_sensor)
```

[156]:
```
print("\nTotal Traffic per Sensor:", total_per_sensor)
print("Busiest Sensor (0-5):", busiest_sensor)
```

```
Total Traffic per Sensor: [1385 1203 1149 1240 1376 1280]
Busiest Sensor (0-5): 0
```

Threshold and Output:

[158]:
```
# Apply threshold (cap at 50 cars/min)
capped = np.clip(data, 0, 50)
print("\nCapped Traffic Data:\n", capped)
```

```
Capped Traffic Data:
 [[19 38 50 18 50 32]
 [50 34  6 50 50  5]
 [38 50 50 32 40 50]
 [50 36 50 50 50 50]
 [50 50 50 50 50 50]
 [23 50 24 50 50 50]
 [50 10 50 24  2 50]
 [50 50 50 42 40 50]
 [47  3 20  0 48 50]
 [50 50 50 50 30 50]
 [11 46 50 41 50 50]
 [50 10 50  9 50  4]
 [42  3 42 50 48 18]
 [18 12 32 33 50  2]
 [47 50 22 50 22 50]
 [50 50 50 22 50 34]
 [50 40 15 50 50 37]
 [48  4 50 29 50 50]
 [48 50  9 50 50 48]
 [50 50 27 50 50 28]
 [50 50 29 25 50  6]
 [50 50 14 50  5 50]
 [17 50 26 50 50 50]
 [50 50 50 50 43 50]]
```

[160]:
```
# Compute total energy usage
energy_usage = np.sum(capped * 0.5)
print("\nTotal Energy Usage:", energy_usage)
```

```
Total Energy Usage: 2813.5
```

Question 10: You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

F = [[5,2,1],

[3,1,4],

[6,3,2],

[4,2,5],

[7,1,3]]

Transformation matrix:

T = [[2,1,0],

[1,3,1],

[0,2,2]]

Task:

- **Compute transformed forces: F_new = F × T**
- **Compute magnitude of each force vector**
- **Identify the component experiencing highest transformed force**
- **Perform eigenvalue decomposition of T**

Code:

PROGRAM 10

[163]:
```
F = np.array([[5,2,1],
              [3,1,4],
              [6,3,2],
              [4,2,5],
              [7,1,3]])
T = np.array([[2,1,0],
              [1,3,1],
              [0,2,2]])
```

[165]:
```
# Transformed forces
F_new = F.dot(T)
print("F_new:\n", F_new)
```

```
F_new:
 [[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
```

[167]:
```
# Magnitudes
mags = np.linalg.norm(F_new, axis=1)
print("\nMagnitudes:", mags)
```

```
Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
```

```
Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
```

[169]:
```
# Busiest component (highest magnitude)
busiest_idx = np.argmax(mags)           # zero-based index
print("\nBusiest component index (0-based):", busiest_idx)
print("Busiest component (1-based):", busiest_idx+1)
print("Max magnitude:", mags[busiest_idx])
```

```
Busiest component index (0-based): 3
Busiest component (1-based): 4
Max magnitude: 25.37715508089904
```

[171]:
```
# Eigen decomposition of T
eigvals, eigvecs = np.linalg.eig(T)
print("\nEigenvalues:\n", eigvals)
print("\nEigenvectors (columns correspond to eigenvalues):\n", eigvecs)
```

Output:

```
Eigenvalues:
 [4.30277564 2.         0.69722436]

Eigenvectors (columns correspond to eigenvalues):
 [[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.70737023e-17 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

LAB NO 11

VISUALIZATION & MACHINE LEARNING TASKS

Machine Learning in AI

- **Definition**: Machine Learning is a subset of AI focused on algorithms that improve automatically through experience and data.
- **Goal**: To allow computers to learn from historical data and make accurate predictions or classifications.
- **Difference from Traditional Programming**: Instead of hardcoding rules, ML models infer rules from data.

Types of Machine Learning

Supervised Learning

- Trains on labeled data (input + output).
- **Example:** Predicting house prices based on features.
- **Algorithms:** Linear Regression, Decision Trees, Support Vector Machines.

Unsupervised Learning

- Works on unlabeled data to find hidden patterns.
- **Example:** Customer segmentation using clustering.
- **Algorithms:** K-Means, Hierarchical Clustering, PCA.

Reinforcement Learning

- Learns through trial and error with rewards/penalties.
- **Example:** Training robots or game-playing agents.

Semi-Supervised & Self-Supervised Learning

- Mix of labeled and unlabeled data.
- Useful when labeling is expensive.

Key Concepts

- **Dataset**: Collection of examples used for training/testing.
- **Features**: Input variables (e.g., age, income).
- **Labels**: Output variable (e.g., churn = yes/no).
- **Model**: Mathematical representation of learned patterns.

- **Training**: Process of fitting the model to data.
- **Evaluation**: Measuring accuracy, precision, recall, etc.

Basic Syntax in Python (Scikit-learn Example)

Supervised Learning (Linear Regression)

```
[85]: from sklearn.linear_model import LinearRegression
      import numpy as np

      # Example dataset
      X = np.array([[1], [2], [3], [4], [5]])   # Features
      y = np.array([2, 4, 6, 8, 10])            # Labels

      # Train model
      model = LinearRegression()
      model.fit(X, y)

      # Predict
      prediction = model.predict([[6]])
      print("Predicted value:", prediction)
```

Output:

```
Predicted value: [12.]
```

Classification (Logistic Regression)

```
[87]: from sklearn.linear_model import LogisticRegression

      X = [[1], [2], [3], [4]]   # Features
      y = [0, 0, 1, 1]           # Labels

      model = LogisticRegression()
      model.fit(X, y)

      print("Prediction for 5:", model.predict([[5]]))
```

Output:

```
Prediction for 5: [1]
```

Applications of Machine Learning

- **Healthcare**: Disease prediction, patient monitoring.
- **Business**: Sales forecasting, fraud detection.
- **Education**: Student performance prediction.
- **Technology**: Recommendation systems, chatbots, self-driving cars.

Challenges & Risks

- **Data Quality**: Poor data → poor predictions.
- **Bias**: Models can inherit biases from training data.
- **Interpretability**: Complex models (like deep learning) are hard to explain.
- **Scalability**: Large datasets require significant computational resources.

LAB EXERCISE

Task 1: Patient Stay Analysis (Healthcare)

You are given a dataset containing age, disease_type, and hospital_stay_days. Write a Python program using Seaborn to:

- **Plot the distribution of hospital stay days**
- **Compare hospital stay across different disease types**
- **Identify which disease has the highest median stay**

Code:

PATIENT STAY ANALYSIS(HEALTH CARE)

[1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

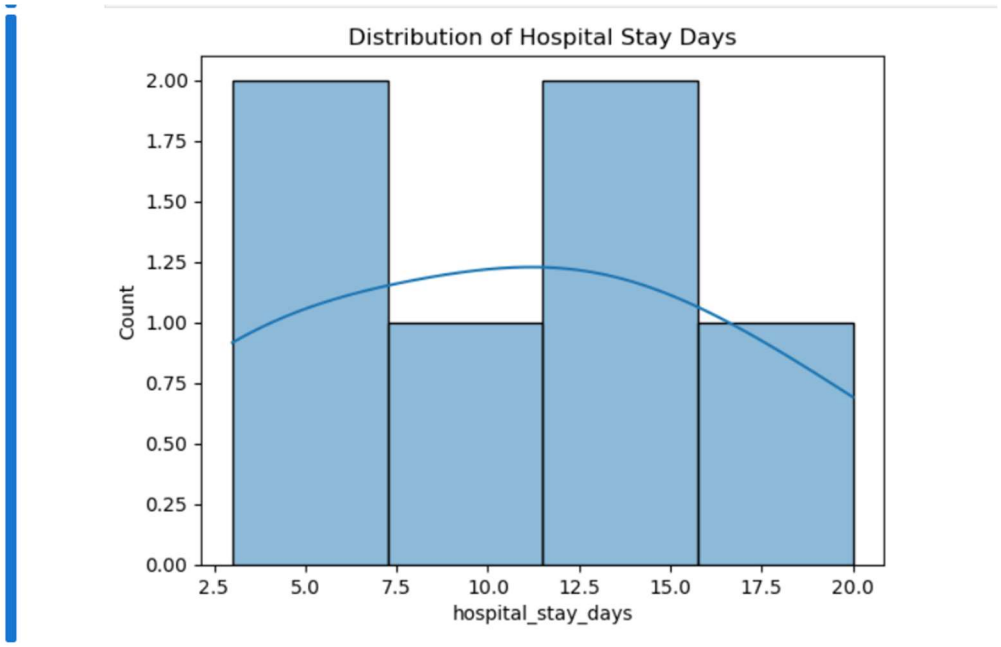
[3]:
```
# Example dataset
data = pd.DataFrame({
    "age": [25, 40, 60, 35, 50, 70],
    "disease_type": ["Flu", "Cancer", "Diabetes", "Flu", "Cancer", "Diabetes"],
    "hospital_stay_days": [3, 15, 10, 4, 20, 12]
})
```

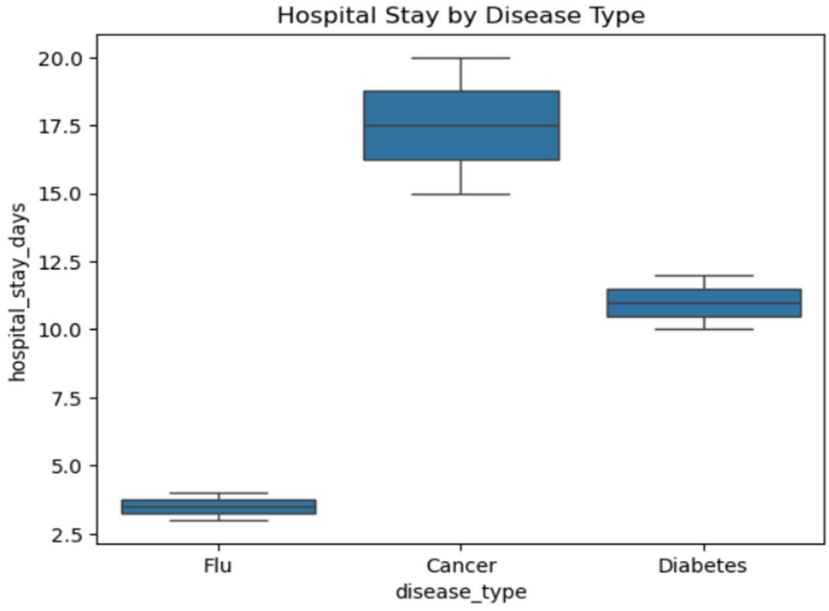
[5]:
```
# Distribution plot
sns.histplot(data["hospital_stay_days"], kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.show()
```

[7]:
```
# Compare across disease types
sns.boxplot(x="disease_type", y="hospital_stay_days", data=data)
plt.title("Hospital Stay by Disease Type")
plt.show()
```

[9]:
```
# Identify highest median stay
median_stays = data.groupby("disease_type")["hospital_stay_days"].median()
print("Disease with highest median stay:", median_stays.idxmax())
```

Output:





```
Disease with highest median stay: Cancer
```

Task 2: Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.
Write code to:

- **Visualize sales distribution per region using Seaborn**
- **Detect regions with high sales variability**
- **Label the axes and add a title**

Code:

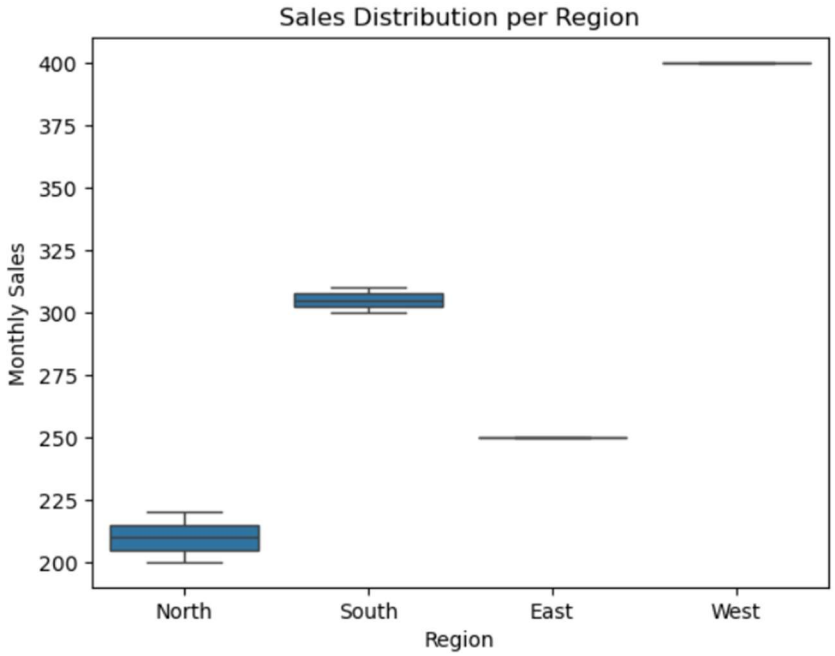
SALES DISTRIBUTION COMPARISON (BUSINESS)

[13]:
```
data = pd.DataFrame({
    "region": ["North", "South", "East", "West", "North", "South"],
    "monthly_sales": [200, 300, 250, 400, 220, 310]
})
```

[15]:
```
sns.boxplot(x="region", y="monthly_sales", data=data)
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.title("Sales Distribution per Region")
plt.show()
```

[17]:
```
# Detect variability
variability = data.groupby("region")["monthly_sales"].std()
print("Regions with high variability:\n", variability.sort_values(ascending=False))
```

Output:




```
Regions with high variability:
 region
North    14.142136
South     7.071068
East           NaN
West           NaN
Name: monthly_sales, dtype: float64
```

Task 3: Student Performance Correlation (Education)

A dataset contains study_hours, attendance, and final_score .
Using Seaborn:

- **Create a pair plot**
- **Plot a correlation heatmap**
- **Interpret which feature most affects the final score**

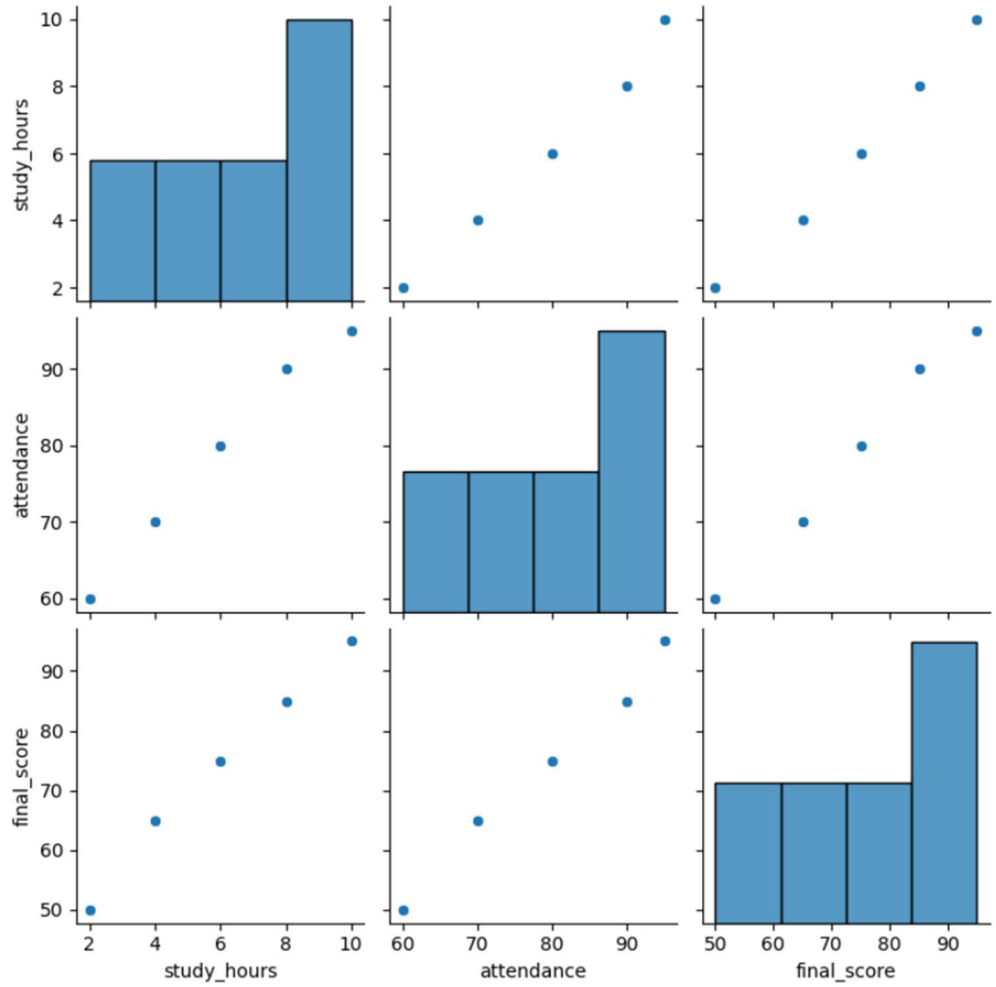
Code:

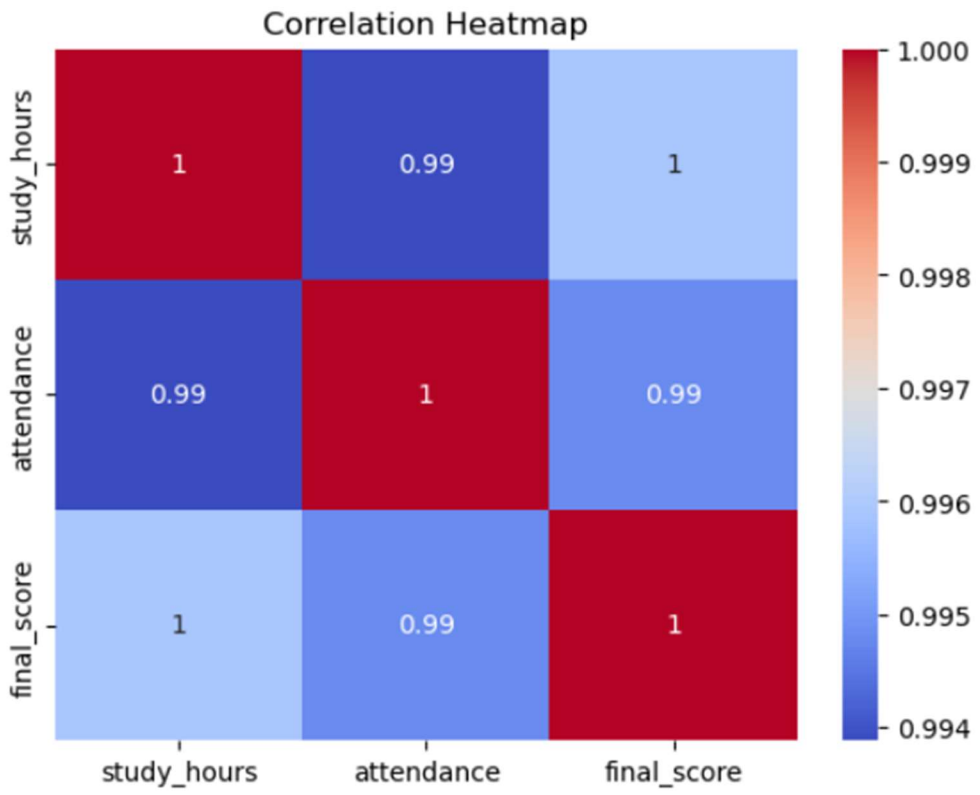
STUDENT PERFORMANCE CORRELATION (EDUCATION)

[20]:
```
data = pd.DataFrame({
    "study_hours": [2, 4, 6, 8, 10],
    "attendance": [60, 70, 80, 90, 95],
    "final_score": [50, 65, 75, 85, 95]
})
```

[22]:
```
# Pair plot
sns.pairplot(data)
plt.show()
```

Output:






```
Feature most affecting final score: study_hours
```

Task 4: Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.
Write a program to:

- **Visualize relationships using Seaborn scatter plots**
- **Color-code points based on spending score**
- **Identify potential customer segments visually**

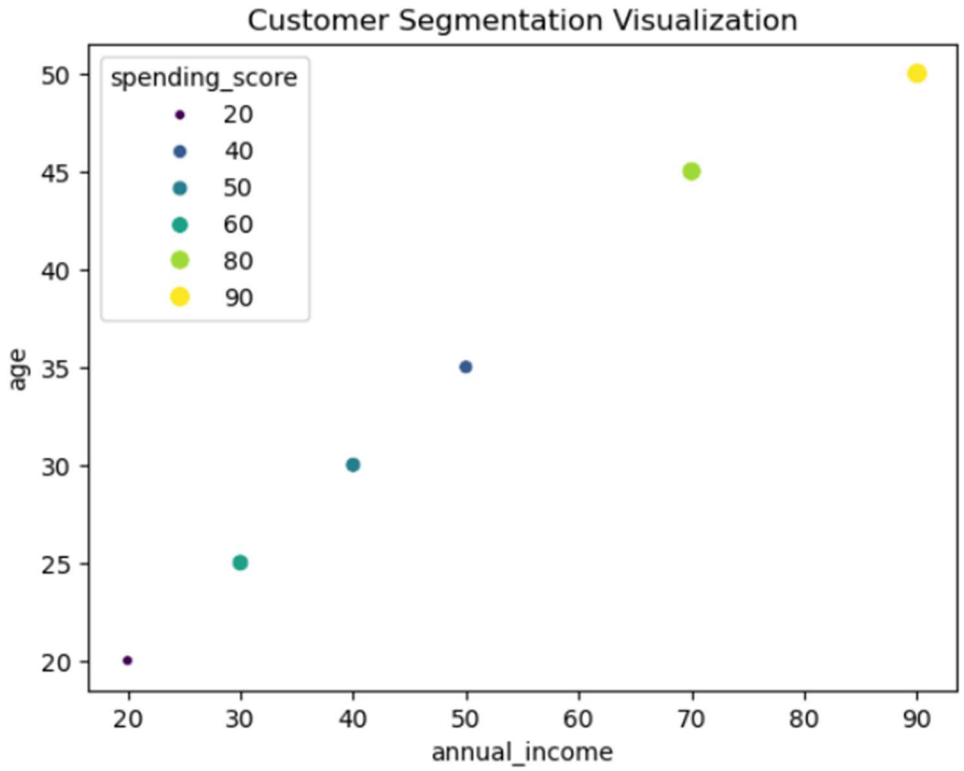
Code:

CUSTOMER SEGMENTATION VISUALIZATION

```
[83]: data = pd.DataFrame({
          "age": [25, 35, 45, 20, 30, 50],
          "annual_income": [30, 50, 70, 20, 40, 90],
          "spending_score": [60, 40, 80, 20, 50, 90]
      })

      sns.scatterplot(x="annual_income", y="age", hue="spending_score", size="spending_score", data=data, palette="viridis")
      plt.title("Customer Segmentation Visualization")
      plt.show()
```

Output:



Task 5: Model Result Visualization

You trained a classifier and obtained y_true and y_pred.
Write code using Seaborn to:

- **Plot a confusion matrix heatmap**
- **Add annotations and color bar**
- **Clearly label predicted vs actual classes**

Code:

MODEL RESULT VISUALIZATION

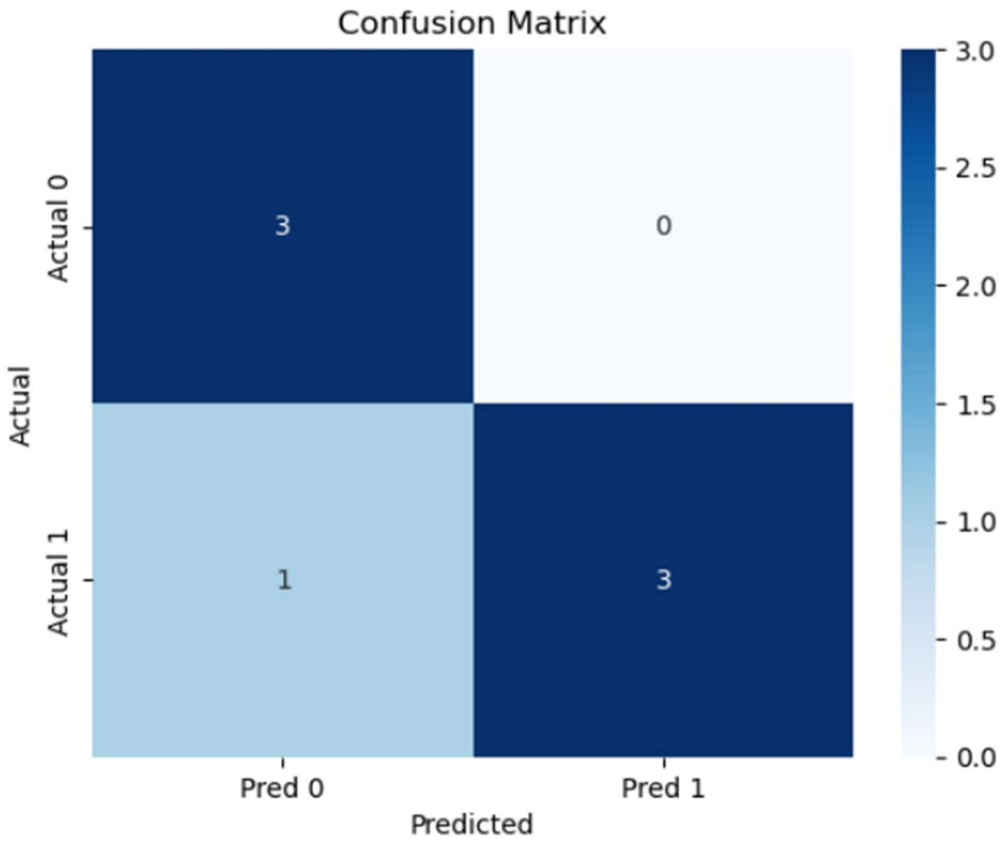
```
[27]: from sklearn.metrics import confusion_matrix

y_true = [0, 1, 0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 0, 1, 1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Pred 0", "Pred 1"], yticklabels=["Actual 0", "Actual 1"])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Output:


Confusion Matrix
Actual 0
Actual 1
Actual
3
0
1
3
Pred 0
Pred 1
Predicted
3.0
2.5
2.0
1.5
1.0
0.5
0.0

LAB NO 12

LAB EXERCISE

Task 6: Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.

Write a Scikit-learn program to:

- **Encode categorical variables**
- **Train a Logistic Regression model**
- **Evaluate using accuracy and confusion matrix**

Code:

ML TASKS (Sci-kit Learn)

[62]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

[64]:
```
data = pd.DataFrame({
    "tenure": [1, 5, 10, 2, 8],
    "monthly_charges": [20, 50, 70, 30, 90],
    "contract_type": ["Month-to-month", "One year", "Two year", "Month-to-month", "One year"],
    "churn": [1, 0, 0, 1, 0]
})
```

[66]:
```
# Encode categorical
le = LabelEncoder()
data["contract_type"] = le.fit_transform(data["contract_type"])
```

[68]:
```
X = data[["tenure", "monthly_charges", "contract_type"]]
y = data["churn"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Output:

```
Accuracy: 0.5
Confusion Matrix:
 [[1 1]
 [0 0]]
```

Task 7: House Price Prediction

A dataset contains area, bedrooms, location_score, and price.
Implement a regression model to:

- **Split data into train/test sets**
- **Train Linear Regression**
- **Predict prices and calculate Mean Squared Error**

Code:

HOUSE PRICE PREDICTION

[70]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = pd.DataFrame({
    "area": [1000, 1500, 2000, 2500, 3000],
    "bedrooms": [2, 3, 3, 4, 4],
    "location_score": [7, 8, 9, 6, 10],
    "price": [200000, 250000, 300000, 350000, 400000]
})
```

[74]:
```
X = data[["area", "bedrooms", "location_score"]]
y = data["price"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

Output:

```
Mean Squared Error: 0.0020118281272970967
```

Task 8: Disease Classification System

Patient data includes glucose, BMI, age, and outcome.
Write code to:

- **Normalize the data**

- **Train a Decision Tree classifier**
- **Evaluate precision, recall, and F1-score.**

Code:

DISEASE CLASSIFICATION SYSTEM

[44]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

data = pd.DataFrame({
    "glucose": [120, 150, 80, 200, 90],
    "BMI": [25, 30, 28, 35, 22],
    "age": [40, 50, 30, 60, 25],
    "outcome": [1, 1, 0, 1, 0]
})
```

[46]:
```
X = data[["glucose", "BMI", "age"]]
y = data["outcome"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

Output:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

Task 9: Email Spam Detection

You are given email text and spam labels.
Using Scikit-learn:

- **Convert text to numerical features (TF-IDF)**
- **Train a Naive Bayes classifier**
- **Test the model on new email text**

Code:

EMAIL SPAM DETECTION

[76]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

data = pd.DataFrame({
    "email_text": ["Win money now", "Meeting tomorrow", "Cheap loans available", "Project deadline"],
    "spam": [1, 0, 1, 0]
})
```

[80]:
```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(data["email_text"])
y = data["spam"]

model = MultinomialNB()
model.fit(X, y)

# Test on new email
new_email = ["Exclusive offer just for you"]
X_new = vectorizer.transform(new_email)
print("Prediction (1=Spam, 0=Not Spam):", model.predict(X_new)[0])
```

Output:

```
Prediction (1=Spam, 0=Not Spam): 0
```

Task 10: Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.
Write a program to:

- **Build a Random Forest classifier**
- **Predict whether a student is "At Risk"**
- **Display feature importance**

Code:

STUDENT RISK PREDICTION

[54]:
```
from sklearn.ensemble import RandomForestClassifier

data = pd.DataFrame({
    "attendance": [80, 60, 90, 50, 70],
    "mid_marks": [40, 30, 50, 20, 35],
    "final_marks": [60, 45, 70, 30, 50],
    "risk": ["No", "Yes", "No", "Yes", "Yes"]
})
```

[56]:
```
X = data[["attendance", "mid_marks", "final_marks"]]
y = data["risk"]

model = RandomForestClassifier()
model.fit(X, y)

print("Feature Importance:", model.feature_importances_)
```

Output:

```
Feature Importance: [0.33333333 0.39583333 0.27083333]
```